

Dr. Wittawin Susutti

wittawin.sus@kmutt.ac.th

CSS234 WEB PROGRAMMING II

LECTURE 08 SECURITY & TESTING

What is OWASP?

Global Standard

- OWASP (Open Web Application Security Project) is a non-profit foundation dedicated to improving software security.

The Top 10 List

- A consensus-based document representing the most critical security risks to web applications globally.

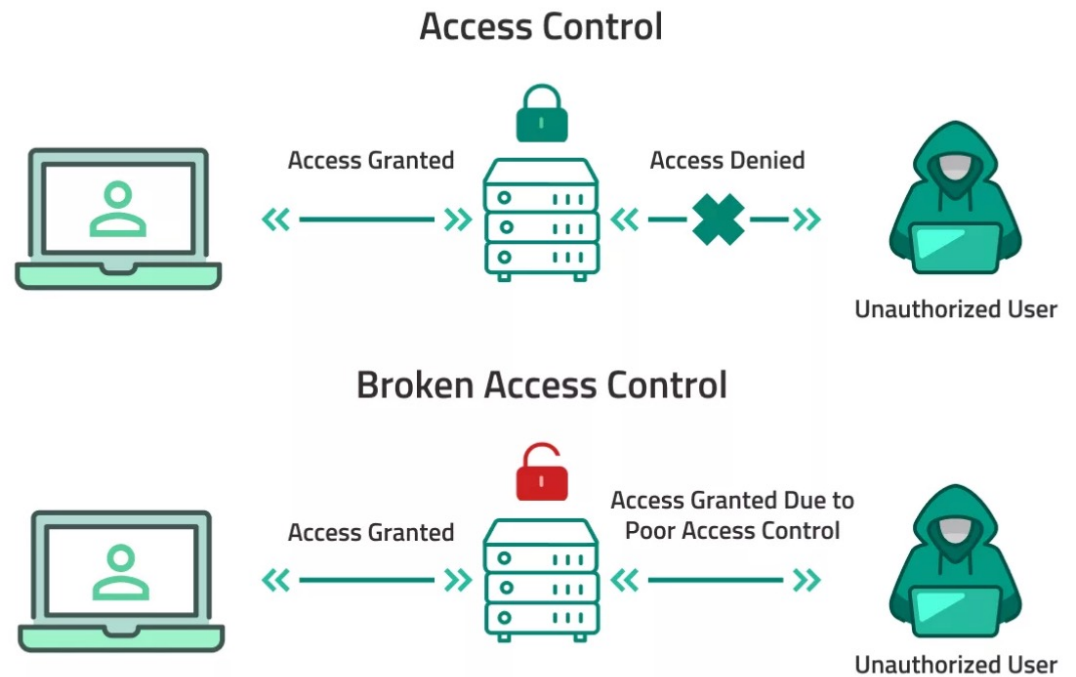
Broken Access Control

Unauthorized Entry

- Allowing users to access resources or data outside their intended permissions.
 - Bypassing access checks.
 - Modifying URLs or internal state.
 - Elevating privileges without auth.

Prevention

- Deny by default and use least-privilege principles.



Cryptographic Failures

Exposing Sensitive Data

- Formerly "Sensitive Data Exposure."
- It focuses on the failure to protect data at rest and in transit.

Common Pitfalls

- Use of weak ciphers (MD5, SHA1), cleartext transmission (HTTP), and unencrypted PII.

Injection

Untrusted Data Handling

- Occurs when malicious data is sent to an interpreter as part of a command or query.
- This includes SQL, NoSQL, and Command Injection.

Impact

- Attackers can read sensitive data, modify database content, or execute admin operations.

Solution

- Use parameterized queries and input validation.

Insecure Design

Architectural Flaws

- Focuses on risks related to design flaws.
- Even a perfect implementation cannot fix an insecure architecture.

Threat Modeling

- Prevention requires threat modeling and secure design patterns during the earliest stages of SDLC.

Shift Left

- Integrate security into the design phase.
- A "security-by-design" mindset is mandatory.

Security Misconfiguration

Defaults

- Leaving default passwords or unnecessary features enabled on your servers.

Cloud Errors

- Open S3 buckets or misconfigured permissions in cloud infrastructure.

Vulnerable Components

Supply Chain Risk

- Using libraries, frameworks, or dependencies that have known security vulnerabilities.
 - Outdated OS or web servers.
 - Vulnerable third-party NPM/Pip packages.
 - Lack of patching strategy.

Auth Failures

Broken Authentication

- Incorrectly implemented identity, authentication, and session management functions.
 - Permitting brute-force attacks.
 - Weak password requirements.
 - Lack of Multi-Factor Auth (MFA).
 - Session hijacking vulnerabilities.

The Remaining Critical Risks

Software & Data Integrity Failures

- Insecure CI/CD pipelines and untrusted auto-updates.

Security Logging & Monitoring

- Inability to detect or respond to breaches in real-time.

Server-Side Request Forgery (SSRF)

- Forcing the server to fetch a malicious URL or internal resource.

AuthN vs AuthZ

Authentication

- Verifying **Identity**.
- Are you who you say you are? (e.g., Login, Passwords, Biometrics).

Authorization

- Verifying **Permissions**.
- What are you allowed to do? (e.g., Admin vs User roles).

The Difference

- AuthN is the door to the house.
- AuthZ is the permission to open specific drawers inside.

RBAC & ABAC

RBAC (Role-Based)

- Access is granted based on predefined roles (e.g., Admin, Editor, Viewer).
- Static and easy to audit.
- Simple for stable orgs.
- Can lead to "Role Explosion."

ABAC (Attribute-Based)

- Access is granted based on attributes (User type, Device, Location, Time).
- Dynamic and Highly flexible for edge cases.
- Complex to implement.

The Zero Trust Mindset

"Never Trust, Always Verify.

Ensure that every request is fully authenticated and authorized before granting access."

JSON Web Tokens (JWT)

Header

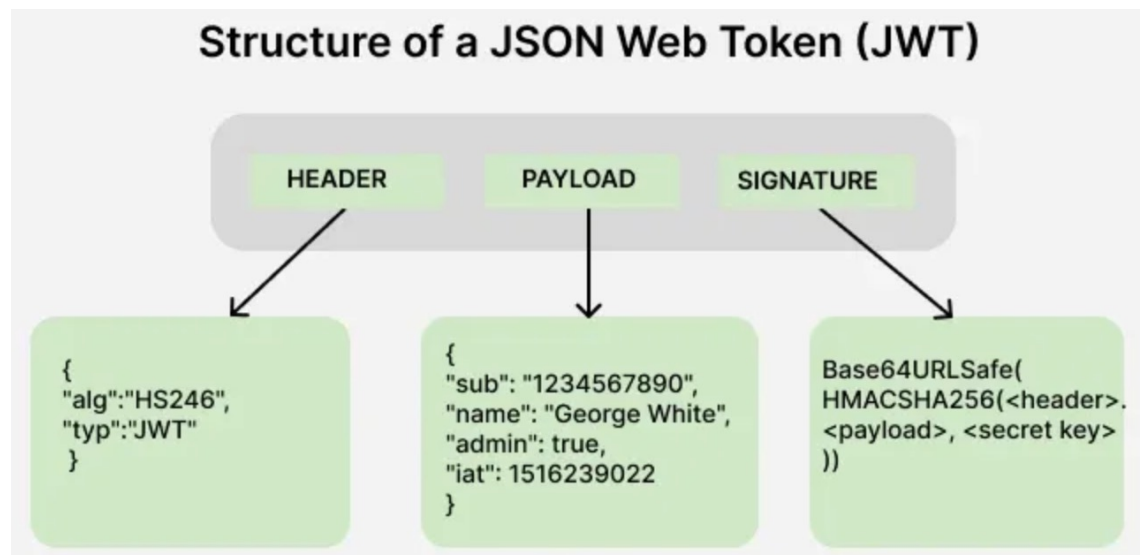
- Token type & signing algorithm.

Payload

- Claims or user information (Encoded, not encrypted).

Signature

- Verification mechanism using a secret key.



Common JWT Vulnerabilities

The "None" Algorithm

- Attackers can change `alg` to `none` to bypass signature verification.

Weak Signing Keys

- Brute-forcing weak secrets using tools.

KID Injection

- Manipulating the Key ID header to point to a malicious key.

Algorithm Confusion

- Forcing the server to use HMAC (Symmetric) to verify an RSA (Asymmetric) public key.

Revocation

JWTs are **stateless**.

- Once issued, they are valid until they expire.
- How do you "Log Out" or "Ban" a user?

Short Expiry (TTL)

- Reduce the window of risk.

Trade-off

- Revocation adds state back to your stateless system.
- Choose wisely based on your security needs.

Refresh Tokens

- Store a "Session ID" in a DB to revoke long-term access.

Deny List

- Store revoked tokens in a fast cache (Redis) until they expire.

JWT Security Best Practices

Algorithm Choice

- Prefer asymmetric RS256 over HS256 for better key management in distributed systems.

Secret Safety

- Never share your signing secret.
- Use long, complex, and environment-stored keys.

Token Expiry

- Keep access tokens short-lived.
- Implement Refresh Tokens for long-term sessions.

Malicious Input

- Applications often assume users will provide "clean" data.
- Attackers exploit this trust via
 - SQL Injection - Manipulating DB queries.
 - XSS - Injecting scripts into the browser.
 - Buffer Overflow - Sending data larger than expected.

GIGO Principle

- "Garbage In, Garbage Out" - but in security, it's "Garbage In, System Down."

Validation vs. Sanitization

Criteria	Input Validation	Input Sanitization
Goal	Is the data correct ?	Is the data safe ?
Action	Reject if invalid.	Clean or modify the data.
Example	Check if email contains "@".	Remove <script> tags.
Analogy	Checking ID at the club door.	Removing weapons before entry.

Validation Strategies

Allow-listing (White-listing)

- Only allow known good patterns (e.g., regex for phone numbers).
- Everything else is rejected.

Block-listing (Black-listing)

- Rejecting known bad patterns.
- Weakest approach because attackers always find new ways to bypass list

Sanitization Techniques

When you must accept complex data (like HTML comments), you need to clean it

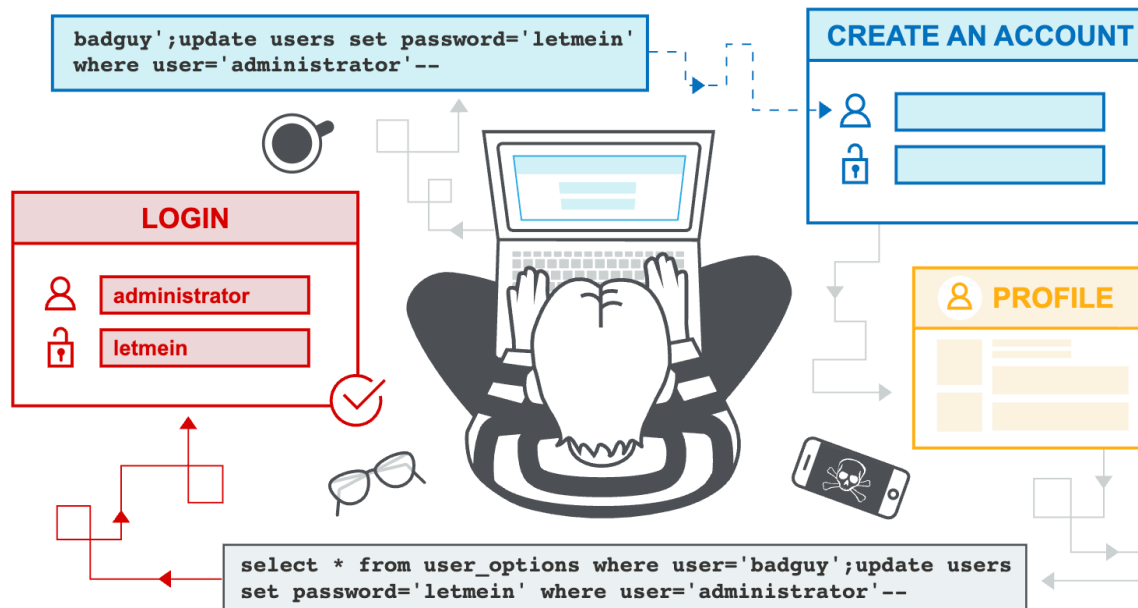
- Stripping - Completely removing tags.
- Escaping - Transforms special characters so the browser treats them as text rather than code.
- Type Casting - Forcing input into a specific type.

Use Proven Libraries

- Don't write your own RegEx for sanitization!
- Use libraries like DOMPurify (Frontend) or sanitize-html (Backend).

SQL Injection

- A technique where an attacker inserts malicious SQL statements into entry fields for execution.
- Use Parameterized Queries (Prepared Statements) or ORM libraries instead of raw string concatenation.



The Golden Rules

Rule #1

- Validate at the Server.
- Client-side validation is for UX; Server-side is for Security.

Rule #2

- Output Encoding.
- Sanitize on input but Encode on output when rendering to HTML.

Rule #3

- Defense in Depth.
- Use Validation, Sanitization, and Parameterized Queries together.

What is Sensitive Data Exposure?

It occurs when an application unintentionally reveals sensitive information to unauthorized parties.

Not just "Hacking"

- Often caused by mistakes in configuration or coding.

Cryptographic Failure

- Weak or missing encryption is the #1.

Legal Risk

- Violates regulations.

The "Crown Jewels"

If this data is exposed, your users and business are at high risk

- **PII (Personally Identifiable Information)** - Name, SSN, Address, Email
- **Financial** - Credit Card numbers, Bank details
- **PHI (Protected Health Information)** - Medical records, Health status
- **Credentials** - Passwords, API Keys, Secrets
- **Intellectual** - Trade secrets, Source code
- **Tracking** - Real-time location, IP logs

Why Does it Happen?

Data in Cleartext

- Transmitting sensitive data over HTTP instead of HTTPS.

Weak Hashing

- Using outdated algorithms like MD5 or SHA-1 for passwords.

Improper Storage

- Storing secrets in .git or plain text config files.

Insecure Headers

- Missing Cache-Control or HSTS.

Classic Mistake

- "I'll just hash the password with a simple function and no salt. No one will find it." —
The hacker found it in 3 seconds.

The Two Pillars of Defense

Data in Transit

- Protection while moving through the network.
- **Solution:** Use TLS 1.2+ (HTTPS).
- **Enforcement:** Use HSTS (HTTP Strict Transport Security).

Data at Rest

- Protection while stored in databases or disks.
- **Solution:** Use AES-256 encryption.
- **Management:** Secure your encryption keys (KMS).

What are Env Vars?

The Definition

- Dynamic key-value pairs stored outside the application code.
- They inform the application about its environment (Dev, Staging, or Production).

The Format

- Standard naming follows the KEY=VALUE pattern.
- Usually written in UPPER_CASE to distinguish from code variables.

Three Core Benefits

Security

- Keeps API keys and DB credentials out of source code.

Portability

- Code runs everywhere; only the configuration changes.

Flexibility

- Easily toggle features or change API endpoints without rebuilding.

The .env File

Local Storage

- A plain text file placed at the project root.
- It serves as a local dictionary for your application's private settings.
- It acts as the single source of truth for secrets during development.

Integrating with Code

Dotenv Library

- The most common way to load .env files into your application's environment.

Process.env

- In Node.js, variables are accessed via the global process.env object.

Consistency

- Ensure the keys in your .env file exactly match your code calls to avoid crashes.

The .gitignore Rule

Preventing Leaks

- NEVER commit your .env file to Git.
- This is the #1 cause of credential theft on platforms like GitHub.

Pro Tip

- Create a .env.example with placeholder values to show other developers what keys they need to set up.

The Testing Pyramid

Unit Tests

- Test individual functions.

Integration Tests

- Test components working together.

E2E Tests

- Test the entire user flow.

